Generative AI with Python — Complete Study Material

Python 3.13+ | Google Gemini | Ollama | LangChain | ChromaDB | Streamlit

---

Table of Contents

1. Python Foundations
2. Generative AI and Large Language Models
3. Prompt Engineering
4. LangChain Framework
5. Streamlit — Building AI Web Apps
6. Embeddings and Vector Databases
7. RAG — Retrieval Augmented Generation
8. Deployment

---

1. Python Foundations

1.1 Virtual Environments

When you install a package with `pip`, it installs globally — meaning it is available to every Python script on your machine. This quickly causes problems: Project A needs `langchain==0.2`, Project B needs `langchain==0.3`, and the two versions are incompatible. Installing one breaks the other.

A **virtual environment** solves this by creating an isolated copy of Python for each project. Every project gets its own folder containing its own Python interpreter and its own set of installed packages. Installing or removing packages inside one environment does not affect any other project or the system Python installation.

Think of it like a clean room — before you enter, you put on a fresh suit. Anything that happens inside stays inside.

```
python -m venv .venv                    # create
.venv\Scripts\Activate.ps1              # activate (Windows)
source .venv/bin/activate               # activate (Mac/Linux)
pip install google-genai python-dotenv # installs only into this env
deactivate                              # exit when done
```

The folder `.venv` is named with a dot so it stays hidden in file explorers, and it is always listed in `.gitignore` — a virtual environment can be hundreds of megabytes and must never be pushed to version control.

1.2 API Keys and the .env File

An API key is a unique identifier that authenticates your code with a cloud service. If someone else obtains your key, they can make API calls billed to your account. This is why keys must never appear in source code, never be committed to git, and never be pasted in chat or email.

The standard practice is to store keys in a `.env` file at the root of your project — a plain text file of `KEY=VALUE` pairs that lives only on your machine.

```
# .env — never commit this file
GEMINI_API_KEY=AIzaSy...your_key_here
```

In Python, the `python-dotenv` library reads this file and makes its contents available through `os.getenv()`. The critical detail: `load_dotenv()` must be called **before** any `os.getenv()` call. It populates the runtime environment; nothing is accessible before that call.

```
from dotenv import load_dotenv
import os

load_dotenv()                              # reads .env into os.environ
api_key = os.getenv("GEMINI_API_KEY")  # now accessible
```

1.3 Variables, Data Types, and f-strings

Python is dynamically typed — the type is inferred from the assigned value, not declared by you. The most common types in AI code are strings (text), floats (decimals like temperature), integers (token counts), booleans (True/False), and NoneType (the absence of a value).

Passing the wrong type to an API causes errors that can be hard to trace — for example, `"0.7"` (a string) instead of `0.7` (a float) as temperature. Type hints in function signatures don't enforce types at runtime, but they document intent and help IDEs catch mistakes early.

f-strings are the standard way to compose prompts dynamically. They evaluate any Python expression inside `{}` at runtime:

```
topic = "embeddings"
limit = 150
prompt = f"Explain {topic} in under {limit} words."
```

For multi-line prompts — the standard pattern in RAG systems — triple-quoted f-strings are used:

```
prompt = f"""Answer only from the context below.

Context:
{context}

Question: {question}""".strip()
```

`.strip()` removes the leading newline created by the opening triple-quote on its own line.

1.4 Lists and Dictionaries

These two structures underpin every part of an LLM application.

Lists are ordered, mutable sequences. Chat history is a list of messages. Document chunks are a list of strings. Retrieved results come back as a list. A list's order matters — the model sees messages in the sequence they appear.

Dictionaries are key-value mappings. Every message in a conversation is a dictionary with `"role"` and `"content"` keys. Every API configuration is a dictionary. The most important habit: use `.get()` instead of direct key access. Direct access (`msg["role"]`) raises `KeyError` if the key is missing; `.get("role", "unknown")` returns the default safely.

```
# Standard chat history format used by every major LLM API
history = [
    {"role": "system",    "content": "You are a helpful assistant."},
    {"role": "user",      "content": "What is RAG?"},
    {"role": "assistant", "content": "RAG combines retrieval with generation."},
]
```

List comprehensions create a new list by transforming or filtering another in one line. They appear constantly when processing chunks, filtering documents, or extracting fields from API responses:

```
# Extract only user messages from history
user_msgs = [m["content"] for m in history if m["role"] == "user"]
```

1.5 Functions

Well-designed functions in AI code do one thing — build a prompt, call the API, parse a response, or store a result — and receive everything they need as arguments rather than relying on global variables. This makes them testable, reusable, and easy to reason about.

`*args` collects any number of positional arguments into a tuple. **`**kwargs`** collects any number of keyword arguments into a dictionary. You will encounter both when building wrappers around LLM clients or configuration utilities.

The function signature `def ask(question: str, temperature: float = 0.7) -> str` communicates: this function takes a required string and an optional float, and returns a string. Default arguments let callers specify only what they want to override.

1.6 Classes and Object-Oriented Programming

A class is a blueprint for creating objects. An object combines data (attributes) and behaviour (methods) in a single unit. Every LangChain component — a language model, a retriever, a memory object — is a class you instantiate and configure.

```
# Creating an instance: calling the class like a function
llm = ChatGoogleGenerativeAI(model="gemini-3.8-flash", temperature=0.3)
```

This calls the class's `__init__` constructor with the provided arguments and returns a configured object. You then call methods on it: `llm.invoke("hello")`.

Inheritance allows a class to extend another. `ChatGoogleGenerativeAI` inherits from LangChain's base `BaseChatModel`. This is why it has `.invoke()` even though you never see

that defined in the Gemini-specific code — it's inherited. Understanding this helps you read framework documentation: a class inherits all capabilities of its parent and adds only what's specific to it.

1.7 File I/O

AI applications read data from files (PDFs, text documents, JSON datasets) and write results back. Always use the `with open(...)` context manager — it guarantees the file is closed correctly even if an exception occurs mid-operation.

JSON deserves special attention because LLMs are frequently asked to produce structured JSON, and parsing that output is a common failure point.

- `json.loads()` — parses a JSON **string** into a Python dict or list
- `json.load()` — reads from a **file object**
- `json.dumps()` — converts a dict or list to a JSON string
- `json.dump()` — writes to a file object

Confusing `loads` and `load` (one takes a string, one takes a file) is one of the most common bugs in AI output-processing code. If the LLM produces malformed JSON — a missing brace, a trailing comma — `json.loads()` raises `json.JSONDecodeError`. Every system that relies on JSON from an LLM must handle this.

1.8 Error Handling

In applications that call external APIs, errors are routine, not exceptional. Rate limits get hit. Networks drop. Models return malformed output. Code that does not handle these cases crashes at the worst possible moment in production.

Python's `try / except` block catches specific exception types and lets you respond appropriately — log the error, return a safe default, retry, or show the user a helpful message rather than a raw traceback.

The best practice is to be specific about which exception you catch. `except Exception` catches everything, which is a useful last resort. But catching specific exceptions like `json.JSONDecodeError` or `FileNotFoundError` lets you handle each situation differently and avoids silently swallowing bugs you should be fixing.

```
try:
    result = chain.invoke(question)
except json.JSONDecodeError:
    result = "Could not parse the model's response."
except Exception as e:
    result = f"An error occurred: {e}"
```

---

2. Generative AI and Large Language Models

2.1 The AI Hierarchy

Understanding where Generative AI sits in the broader landscape sets realistic expectations for what these systems can and cannot do.

Artificial Intelligence (AI) is the broadest category — any system that exhibits behaviour we associate with intelligence. Rule-based systems, chess engines, and spam filters are all AI, even though they do not learn.

Machine Learning (ML) is a subset of AI where systems learn patterns from data instead of following hand-written rules. A spam filter that improves as it sees more email is ML; one with a fixed list of banned keywords is not.

Deep Learning (DL) is a subset of ML using multi-layer neural networks. The "deep" refers to many layers between input and output. Deep learning powered breakthroughs in image recognition, speech-to-text, and translation that previous ML approaches could not achieve.

Generative AI is a category of deep learning models trained specifically to *create* new content — text, images, audio, video, or code. Rather than classifying an input into a label, generative models learn the underlying structure of the training data well enough to produce new examples from it.

Large Language Models (LLMs) are generative AI focused on text. They are trained on massive quantities of text and learn to predict what token should come next given all previous tokens. This seemingly simple objective, at sufficient scale, produces systems capable of reasoning, summarising, translating, writing code, and answering questions.

```
AI  →  ML  →  Deep Learning  →  Generative AI  →  LLMs
(broad)                                          (specific)
```

2.2 How LLMs Generate Text

An LLM does not retrieve stored answers from a database and does not search the internet. It **generates** text, one token at a time, where each new token is sampled from a probability distribution computed over the entire vocabulary given all previous tokens.

The generation loop:

1. Your input text is broken into tokens by the tokeniser
2. Each token is converted into a high-dimensional vector (embedding)
3. The Transformer processes the sequence through many layers of **attention**
4. The final layer produces a probability distribution: for every word in the vocabulary, how likely is it to come next?
5. A token is sampled from that distribution (influenced by temperature)
6. The new token is appended to the sequence
7. Steps 3–6 repeat until a stop condition (max tokens or end-of-sequence token)

The Transformer's key innovation is the attention mechanism. When generating each token, attention allows the model to simultaneously consider any previous token in the sequence — regardless of how far back it appears. This is why LLMs can correctly reference something said 10,000 tokens earlier, which earlier sequential models like RNNs fundamentally struggled with.

Emergent capabilities — reasoning, code generation, analogical thinking — were not explicitly trained. They emerged as a consequence of scale: models trained on enough data with enough parameters developed general language understanding that transferred to tasks never seen during training.

2.3 Tokens

A token is the atomic unit the model processes. The tokeniser splits raw text into tokens before the model sees anything. A token is approximately 4 English characters or ¾ of an English word. Numbers, punctuation, and non-English characters typically consume more tokens per character.

Tokens matter for three practical reasons:

1. **Cost:** Cloud APIs charge per token (input + output). A 1,000-word query costs more than a 50-word query.
2. **Context window:** The model can only process a limited number of tokens per call. When you hit the limit, the call fails or content gets truncated.
3. **Speed:** More tokens = more computation = longer response time.

A rough rule of thumb: 1 page of English text ≈ 500–700 tokens. A 100-page document ≈ 50,000–70,000 tokens. Keep this in mind when deciding how much context to inject in a RAG prompt.

2.4 The Context Window

The context window is the maximum number of tokens the model can process in a single API call. Everything the model "sees" during that call — the system prompt, conversation history, retrieved document chunks, the user's question, and the space for the generated response — all counts against this single limit.

When the context window fills, you must decide what to cut. The common strategies are: truncate the oldest messages, summarise the conversation history, or use RAG to include only the most relevant document sections rather than entire documents.

Despite modern models having very large context windows (Gemini 3.8 Flash supports 1 million tokens), blindly stuffing all documents is not the right strategy. Research consistently shows models perform worse on information buried in the middle of a very long context — the "lost in the middle" problem. Focused retrieval of the most relevant chunks consistently outperforms raw context stuffing.

2.5 Temperature and Generation Parameters

Temperature is the most frequently tuned parameter. It controls how deterministic or random the model's sampling process is.

At `temperature=0`, the model always picks the single most probable next token. Run the same prompt ten times and get the same response every time. This is appropriate for structured data extraction, JSON output, code generation, and factual Q&A — tasks where consistency matters more than variety.

At higher temperatures (0.8–1.3), the model samples from a wider portion of the probability distribution, introducing variation. This produces more creative, diverse outputs but also increases hallucination risk and incoherence.

Think of temperature as a dial on statistical variance — not "creativity" in the human sense, but randomness in the sampling process.

Temperature	Character	Best for
0.0	Always picks the highest-probability token	JSON extraction, code, factual Q&A
0.2–0.3	Slight variation, mostly consistent	Summarisation, classification
0.6–0.8	Balanced	General chat, explanations
1.0–1.3	High variance	Brainstorming, creative writing

top_p (nucleus sampling) restricts sampling to the smallest set of tokens whose cumulative probability reaches the `top_p` threshold. At `top_p=0.95`, only tokens that collectively account for 95% of the probability mass are considered. This keeps outputs within the range of plausible continuations while still allowing variation. Temperature and top_p both influence randomness through different mechanisms — tune one at a time.

2.6 What an LLM Cannot Do (By Default)

These limitations are critical to communicate clearly when building applications:

No real-time knowledge. An LLM was trained on a fixed dataset with a cutoff date. It knows nothing about events after training ended. This is one of the primary motivations for RAG — connecting the model to current or private information.

No persistent memory between calls. Every API call is stateless. The model has no knowledge of previous calls unless you explicitly include that history in the current call's context. Every chatbot application must manually maintain and pass conversation history.

No access to private data. The model knows only what was in its training data — public internet text. It knows nothing about your company's internal documents, your database, or a file you uploaded. Again: RAG is the standard solution.

Hallucination. The model generates the statistically most likely continuation. If the most likely text is factually wrong, the model produces it confidently. There is no internal fact-checking

mechanism. This is especially dangerous in medicine, law, and finance, where a confident wrong answer causes real harm.

2.7 Cloud Models vs Local Models

When building AI applications, you can call a cloud-hosted model via API (Gemini, GPT-4) or run an open-source model locally using Ollama.

Cloud models are accessed via API key. You pay a small per-token fee and receive responses from a large, highly capable model running on powerful remote hardware. The downsides: cost at scale, internet dependency during inference, and privacy concerns (your prompts go to a third party's server).

Ollama lets you download and run open-source models like `phi3:mini` or `gemma2:2b` entirely on your laptop. No API key. No cost per call. No internet required at inference time. Complete privacy — the prompt never leaves your machine. The tradeoffs: smaller models are less capable, and a laptop CPU is much slower than a cloud GPU.

LangChain's unified model interface means the choice is a single line in your code:

```
llm = ChatGoogleGenerativeAI(model="gemini-3.8-flash", temperature=0.3)  # cloud
llm = ChatOllama(model="phi3:mini", temperature=0.3)                     # local
```

The rest of the chain — prompts, memory, retrievers, output parsers — stays exactly the same either way.

---

3. Prompt Engineering

3.1 What Prompt Engineering Is

The model does not change between calls. Only the input changes. Prompt engineering is the practice of designing inputs that reliably elicit the outputs your application needs.

The quality gap between a naive prompt and a carefully designed one is often larger than the quality gap between model versions. The exact same model, asked the same factual question in different ways, can produce a correct, confident answer or a hallucinated, confident wrong answer. How you ask matters enormously.

Good prompts share a few characteristics:

- Specific task description (not vague, not open to interpretation)
- Defined output format (structure, length, style)
- Relevant context (background the model needs)
- Explicit constraints (what to do, what not to do)
- The right audience/persona

3.2 Zero-Shot Prompting

A zero-shot prompt gives the model a task description and input with no examples. It relies entirely on patterns the model acquired during training.

Zero-shot works well when the task is common enough that the model has seen many examples of it during training — translation, sentiment classification, summarisation, simple factual questions. It struggles when the required output format is unusual, or the task requires domain-specific knowledge not well-represented in training data.

The format specification in a zero-shot prompt is critical. Without "Reply with one word only," the model might explain its answer in a paragraph — technically correct but unusable for downstream processing.

```
prompt = """Classify the sentiment as Positive, Negative, or Neutral.
Reply with one word only.

Review: "The camera is excellent but the battery drains too fast." """
```

3.3 Few-Shot Prompting

A few-shot prompt provides 2–4 examples of input → desired output before presenting the actual question. The model infers the pattern from the examples and applies it.

Few-shot prompting is most valuable when:

- The output format is unusual or highly specific
- Zero-shot produces inconsistent results
- The task requires a particular style not obvious from a task description alone

Two to four examples is the practical sweet spot. Beyond six, you spend tokens for diminishing returns. The quality of examples matters more than quantity — one clear, representative example beats five mediocre ones.

```
prompt = """Extract product name and price. Format: name | price

Input: "The UltraSound Pro headphones are on sale for ₹2,499."
Output: UltraSound Pro | ₹2,499

Input: "Order the CoolBreeze fan for ₹899."
Output:"""
```

The incomplete final `Output:` line signals to the model exactly where it should continue.

3.4 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting instructs the model to reason step by step before producing a final answer. This consistently improves performance on tasks requiring multi-step reasoning — mathematics, logic puzzles, code debugging, and planning.

The mechanism is subtle but important: generating intermediate reasoning tokens changes the

probability distribution for each subsequent token. A model that writes out "`3 × 10 = 30, then 2 × 15 = 30, total = 60`" before writing "The answer is ₹60" is far more likely to reach the correct answer than one that jumps directly to the answer. The reasoning tokens act as a scaffold.

The four-word trigger that consistently activates CoT reasoning across all major LLMs is: **"Think step by step."**

```
prompt = """A store sells apples for ₹10 and oranges for ₹15.
Ravi buys 3 apples and 2 oranges. What is the total cost?
Think step by step."""
```

For complex reasoning, you can provide an example of the reasoning process itself — showing not just the answer but the full chain of thought that leads to it. This is called few-shot CoT and is the most reliable form for difficult tasks.

3.5 Role Prompting

Assigning the model a persona shifts its vocabulary, tone, depth of explanation, and the assumptions it makes about the reader. The model has learned from text produced by many different kinds of people, and role prompting activates the patterns associated with a specific type of author or expert.

```
"You are a senior ML engineer" → technical, jargon-rich, assumes ML background
"You are a patient teacher explaining to a beginner" → analogies, simple
language
"You are a code reviewer looking for bugs" → structured, critical, specific
```

The same question receives fundamentally different responses depending on the active role. Role prompting is also an effective guardrail: a customer support bot told "You are a support agent for Nova Phones. Only answer questions about Nova Phones products" is far less likely to go off-topic than one given no persona.

3.6 System Prompts vs User Prompts

The conversation format used by all major chat APIs separates messages into three roles:

System prompt — Set by the developer. Defines the model's persona, capabilities, restrictions, and consistent behaviour for the entire session. The end user never sees it directly. It is the most powerful lever for controlling application behaviour.

User prompt — The message from the end user. May be raw and unformatted; your application often preprocesses it (injecting context, sanitising) before it reaches the model.

Assistant — The model's response. Previous assistant messages are included in the conversation history so the model maintains coherence across turns.

A critical antipattern is putting constraints only in the user prompt. Constraints belong in the system prompt because they apply to every turn of the conversation. A system prompt like "Only answer questions about cooking. Politely decline anything else" stays active across the entire session; a

user-prompt note gets treated as a one-time instruction and is often forgotten by the model on the next turn.

3.7 Prompt Templates

In production applications, prompts are not static strings — they contain variables that change with each request. A prompt template is a reusable structure with named placeholders that are filled at runtime.

LangChain's `ChatPromptTemplate` formalises this. It takes a list of `(role, template)` tuples, validates that all required variables are provided before the chain runs, and can be serialised, shared, and versioned independently from the rest of the application code.

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}. Be {tone}."),
    ("human", "{question}"),
])
# call: template.invoke({"role": "Python tutor", "tone": "concise", "question":
"..."})
```

This is cleaner than f-string concatenation at scale, and catches missing-variable bugs at chain construction time rather than silently sending a broken prompt to the model.

---

4. LangChain Framework

4.1 Why LangChain Exists

Building an LLM application from scratch means writing a lot of repetitive glue code: formatting prompts, managing chat history, calling the API, parsing the response, storing memory, retrieving documents. The patterns are the same across nearly every application.

LangChain provides a standardised set of composable building blocks for this work. A language model wrapped in `ChatGoogleGenerativeAI` has the same interface as one wrapped in `ChatOllama` — the rest of your code doesn't change when you swap providers. A retriever built on ChromaDB has the same interface as one built on Pinecone. This **provider-agnostic, composable interface** is the framework's primary value.

The five core components:

Component	**Purpose**
Models	Unified interface to any LLM (Gemini, Ollama, OpenAI, etc.)
Prompts	Reusable templates with variables and validation
Chains	Components connected with the `|` pipe operator
Memory	Stores and injects conversation history

Component	Purpose
	automatically
Tools	External capabilities — search, calculators, APIs

4.2 LCEL and the Pipe Operator

LCEL (LangChain Expression Language) uses the `|` operator to connect components into a chain. Each component receives the output of the one before it and produces input for the one after it. The resulting chain is a first-class Python object that can be invoked, streamed, batched, or composed with other chains.

```
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"topic": "embeddings"})
```

Data flows left to right: `{"topic": "embeddings"}` → prompt template → formatted messages → LLM → `AIMessage` → `StrOutputParser` → plain string.

The chain is **lazy** — nothing executes until you call `.invoke()`. This means you build the chain once (at startup or when the user uploads a document) and call it many times, which is efficient and allows the chain itself to be inspected, tested, and logged as a unit.

Without LCEL, the equivalent code is three separate function calls with intermediate variables. The pipe syntax makes the data flow readable at a glance and enables LangChain to instrument, cache, and stream the entire pipeline automatically.

4.3 Language Model Wrappers

LangChain wraps every major LLM provider behind the same interface. The primary method is `.invoke()`, which takes a prompt or messages list and returns a structured response. `StrOutputParser` at the end of the chain extracts the text content from that response.

The wrapper exists not just for syntactic convenience — it adds streaming callbacks, token counting, run logging, prompt caching, and composability with every other LangChain component. The raw Google SDK is appropriate for simple direct calls; the LangChain wrapper is what allows a retriever's output to flow directly into a prompt which flows directly into the model in a unified chain.

4.4 Output Parsers

LLM output is always raw text. But downstream code frequently needs a specific structure: a Python dictionary, a validated object, a list. Output parsers convert LLM text into the format your application needs, sitting at the end of a chain.

StrOutputParser — extracts the text content from an `AIMessage`. Used in almost every chain where the downstream consumer expects a plain string.

JsonOutputParser — instructs the model to return JSON and then parses that JSON into a Python dictionary. Requires the prompt to explicitly request JSON format, and handles the inevitable cases where the model wraps JSON in markdown code fences.

PydanticOutputParser — the most robust option. You define a Pydantic model specifying the expected schema (field names, types, descriptions), and the parser both validates the output against that schema and provides `get_format_instructions()` — a string you inject into the prompt to tell the model exactly what to produce. If the model's output doesn't match the schema, the parser raises a validation error, making failures explicit rather than silent.

The pattern: schema defines what you expect → format instructions tell the model what to produce → parser validates what arrives. This completes a feedback loop that dramatically improves structured output reliability.

4.5 Conversation Memory

An LLM has no inherent memory between calls. Each API call is stateless — the model knows nothing about previous calls unless you explicitly include that history in the current call's context. LangChain's memory components automate this by tracking conversation history and injecting it into every prompt.

ConversationBufferMemory stores every message verbatim. The complete conversation — every user message and every assistant response — is injected into each new prompt. This preserves all detail and is simple to reason about, but the context window grows with every exchange. For conversations longer than ~20 turns, the prompt size becomes problematic.

ConversationSummaryMemory uses the LLM itself to periodically compress old conversation history into a running summary. Instead of storing "User asked X, assistant said Y, user asked Z...", it stores "The user is building a RAG system. The assistant has explained document loading and chunking so far." Context window size stays approximately constant regardless of conversation length, at the cost of losing verbatim detail.

Choosing between them:

- Customer support conversations that may reference specific order numbers, error messages, or exact quotes → BufferMemory
- Open-ended educational or assistant conversations where the general topic matters more than verbatim history → SummaryMemory

---

4.6 RunnablePassthrough and RunnableParallel

These utilities solve structural challenges in chains that emerge when more than one component needs to receive the same input.

RunnablePassthrough holds a value unchanged while the rest of the chain transforms other values. In a RAG chain, the user's question must reach two places: the retriever (to search for relevant chunks) and the prompt template (to be asked of the LLM). Without RunnablePassthrough, passing the question to the retriever would consume it, leaving nothing for the prompt template.

RunnableParallel runs multiple sub-chains simultaneously with the same input and collects their results into a dictionary. In a RAG application, this means you can generate an answer and retrieve source documents in one call, getting both back at once. This is essential for showing citations alongside answers.

```
chain = RunnableParallel(
    answer=rag_answer_subchain,
    sources=retriever,
)
result = chain.invoke("What is the refund policy?")
# result["answer"]  → the answer string
# result["sources"] → list of Document objects with page/source metadata
```

---

5. Streamlit — Building AI Web Apps

5.1 What Streamlit Is and How It Works

Streamlit turns a Python script into an interactive web application with no HTML, CSS, or JavaScript. You write Python; Streamlit handles all front-end rendering, server management, and browser communication. The result is a shareable web app deployed with one command.

The execution model is unlike traditional web frameworks and must be understood to write correct Streamlit code: **every user interaction causes the entire script to re-run from top to bottom.** When a user clicks a button, the complete `app.py` re-executes. When a user types in an input field, the whole script runs again.

This means any Python variable defined in the script is reset to its initial value on every re-run. A `messages = []` at the top of the file produces an empty list on every button click — making any conversational state disappear instantly.

5.2 Session State — The Solution to Re-runs

`st.session_state` is a dictionary that **persists across re-runs within the same user's browser session**. It is the central persistence mechanism in Streamlit. Any value that must survive a user interaction must live here.

The standard initialisation pattern — check before setting — ensures the value is created once and preserved on every subsequent re-run:

```
if "messages" not in st.session_state:
    st.session_state.messages = []
```

What must go in session_state:

- Conversation message history
- The loaded LangChain chain (so it's built once, not on every keystroke)
- Uploaded file metadata
- Any configuration the user has set

What does not need session_state:

- Values computed fresh from session_state each run (derived state)
- Constants defined at module level
- Values only needed within the current re-run

Expensive objects — LLM clients, vector stores, LangChain chains — should always be stored in session_state. Re-creating a vector store on every keystroke would re-embed all documents every time, making the app unusable.

5.3 Chat Components

Streamlit provides two purpose-built components for conversational UIs:

`st.chat_message(role)` is a context manager that renders a message bubble with a user or assistant avatar. The `role` parameter is `"user"` or `"assistant"`. Everything written inside the `with` block appears inside the bubble.

`st.chat_input(placeholder)` renders a text input fixed at the bottom of the page — the standard chat layout. It returns the user's typed message when they press Enter, and `None` when the input is empty. Because the script re-runs on every submission, you simply check `if user_input:` to know when a new message arrived.

The architectural pattern: re-render the entire message history on every run (so messages appear after re-runs caused by other interactions), then check for new input and process it.

5.4 Application Architecture Patterns

A well-structured Streamlit AI app separates concerns into three areas:

Sidebar — secondary controls: file upload, model selection, configuration sliders, clear-chat button. The sidebar does not clutter the main interaction area.

Initialisation block — runs at the top, checks session_state for expensive objects (the chain), and creates them if missing. This is the "build once, reuse always" pattern.

Chat area — renders history, accepts new input, calls the chain, appends the response to history, and re-renders the new message. This is the main body of the page.

This separation means a user can change a sidebar setting and the chain rebuilds cleanly, without old state bleeding into the new configuration.

5.5 Status and Feedback Components

Streamlit provides typed visual feedback components that communicate system state clearly:

- `st.success("...")` — green box; confirms the operation worked

- `st.error("...")` — red box; communicates failure
- `st.warning("...")` — yellow box; alerts to something the user should know
- `st.info("...")` — blue box; neutral information
- `with st.spinner("...")` — animated spinner during long operations (PDF processing, LLM calls)

These are not cosmetic. A `st.spinner` during a 10-second PDF indexing operation tells the user the app is working, not frozen. A red `st.error` with a specific message is far more helpful than a silent crash. In production AI apps, clear feedback for both success and failure paths is as important as the AI logic itself.

---

6. Embeddings and Vector Databases

6.1 What Embeddings Are

An **embedding** is a numerical representation of meaning. An embedding model converts a piece of text into a fixed-length list of floating-point numbers — a **vector** — that encodes the semantic content of the text in a form computers can compute with.

The defining property of a good embedding model: **texts with similar meaning produce vectors that point in similar directions.** "The cat sat on the mat" and "A cat rested on a rug" would produce vectors very close together. "The stock market crashed" would produce a vector pointing in a completely different direction. This property — semantic proximity in vector space — enables semantic search without any keyword matching.

Embeddings are produced by specialised models distinct from generative models. `text-embedding-004` from Google maps any text to a 768-dimensional vector (768 numbers). The model was trained so that the geometric relationships between vectors reflect the semantic relationships between texts.

This technology underpins modern search engines, recommendation systems, duplicate detection, document clustering, and — most relevantly here — the retrieval step in RAG systems.

6.2 Why Similarity, Not Keyword Matching

Traditional search engines find documents that contain the words in your query. This breaks in many common situations: a query for "cardiac arrest" won't find a document that only uses the word "heart attack," even though they mean the same thing. A query for "how do I revert a commit" won't find documentation titled "undoing changes in git."

Embedding-based similarity search solves this. Both "cardiac arrest" and "heart attack" embed to nearby vectors because the embedding model learned — from reading billions of sentences — that these phrases appear in the same semantic contexts. Querying with one finds documents containing the other.

Cosine similarity is the standard way to measure closeness between two vectors. It measures the angle between them. A value of 1.0 means they point in the same direction (maximum similarity); 0.0 means they are perpendicular (unrelated).

Cosine similarity is magnitude-independent — it cares about direction, not length. This matters because longer documents produce larger-magnitude vectors, and we don't want document length to skew similarity scores.

6.3 Vector Databases — Purpose and Need

Given a user's question, you want to find the most semantically similar chunks from your document collection. The naive approach — embed the question, embed every chunk, compute cosine similarity for every pair, sort and return the top results — works at small scale. For 100 documents it's fine. For 100,000 documents, computing hundreds of thousands of similarity calculations per query takes too long for an interactive application.

A **vector database** builds an index over stored vectors using approximate nearest-neighbour (ANN) algorithms, enabling sub-second search across millions of vectors. The "approximate" aspect means it may occasionally miss a marginally better match — in practice this quality loss is negligible while the speed gain is enormous.

ChromaDB is an open-source local vector database that requires no configuration and stores data either in memory or on disk. For development and small-to-medium production applications, it is an excellent choice. For enterprise scale, distributed systems like Pinecone, Weaviate, or Qdrant provide higher throughput, access control, and cloud storage.

6.4 ChromaDB Concepts

ChromaDB organises data into **collections** — separate namespaces, like tables in a relational database. Querying one collection does not search any other. A multi-user application might give each user their own collection; a knowledge management system might have one collection per topic or document type.

Each item in a collection has:

- **id** — a unique string identifier
- **document** — the original text
- **embedding** — the vector (ChromaDB can generate this automatically, or you pass your own)
- **metadata** — arbitrary key-value pairs (source file name, page number, date, author, category)

Metadata filtering is a powerful feature: you can restrict a query to a subset of documents matching specific metadata criteria. This enables use cases like "only search this user's uploaded documents," "only search policy documents from the last 6 months," or "only search items tagged as category=legal."

Persistent vs in-memory: An in-memory client stores everything in RAM — fast, but data is lost when the process ends. A `PersistentClient` writes data to disk, surviving process restarts. For production applications that index documents once and query many times, persistence is essential.

6.5 Document Loading

Before embedding, you must load documents from their source format and convert them into a standard structure. LangChain's document loaders do this conversion, producing `Document` objects — each with `page_content` (the text) and `metadata` (source, page number, and any other properties).

PyPDFLoader extracts text page by page from a PDF, creating one `Document` per page. The metadata includes the source path and zero-indexed page number. **TextLoader** reads a plain text file as a single document. **DirectoryLoader** recursively loads all files of a specified type from a folder.

The metadata attached to each document at load time becomes the source citation in a RAG response. A retrieved chunk's `metadata["source"]` and `metadata["page"]` tell the user exactly which document and page the answer came from.

6.6 Chunking — Theory and Practice

Documents must be split into smaller pieces before embedding. This is necessary because:

1. **Granularity:** Embedding a 200-page manual as one vector loses all detail. You cannot retrieve "the section about authentication" if the entire manual is a single chunk.
2. **Context window:** The LLM can only accept a limited number of tokens per call. You inject only the relevant chunks, not the whole document.
3. **Retrieval precision:** A small, focused chunk is more semantically coherent and produces a more accurate embedding than a large chunk mixing many topics.

`RecursiveCharacterTextSplitter` is the standard splitter. It tries to split on `\n\n` (paragraphs) first, then `\n` (lines), then spaces, then individual characters — always using the largest natural boundary that keeps the chunk under `chunk_size`. This preserves paragraph and sentence structure as much as possible, unlike naive character slicing.

chunk_overlap causes consecutive chunks to share some characters at their boundary. Without overlap, a key sentence that falls exactly at a chunk boundary may be split across two chunks, incomplete in both. Overlap ensures boundary content appears fully in at least one chunk.

Choosing chunk size involves a tradeoff:

- Too large: retrieves broad sections with much irrelevant content; wastes context tokens
- Too small: loses surrounding context; fragments coherent ideas; retrieves incomplete information

For most prose documents, 300–600 characters with 10–20% overlap is a practical starting point.

Adjust based on the nature of the content: structured technical documentation tolerates smaller chunks; narrative prose benefits from larger ones.

---

7. RAG — Retrieval Augmented Generation

7.1 The Problem RAG Solves

An LLM trained on public internet data knows nothing about your organisation's internal documents, your product database, or a file uploaded by your user today. Two naive approaches to bridge this gap both fail:

Approach 1 — Put everything in the prompt: For a 10-page document, this works. For a 1,000-page technical manual, it exceeds the context window. Even if the context window is large enough, models perform poorly on information buried in the middle of very long contexts (the "lost in the middle" problem). And it's expensive — you pay for all those tokens on every query.

Approach 2 — Fine-tune the model on your data: This is expensive (thousands of dollars for large models), slow (hours to days), and must be repeated every time your data changes. Fine-tuning teaches the model style and patterns — it doesn't reliably inject facts that can be recalled accurately on demand.

RAG solves this by retrieving only the *relevant* sections of your document for each specific question and injecting just those sections into the prompt. The model receives exactly what it needs to answer this question — nothing more.

The name captures the process: **Retrieval** (find relevant chunks) **Augmented** (inject them into the prompt) **Generation** (LLM generates the grounded answer).

7.2 How RAG Works — The Two Phases

Phase 1 — Indexing (offline, run once per document set):

Documents are loaded, split into chunks, each chunk is embedded, and the (chunk, embedding, metadata) triplets are stored in a vector database. This is typically a background process triggered when a new document is uploaded, not when a user asks a question.

Phase 2 — Querying (online, per user question):

1. The user's question is embedded using the same model used for indexing
2. The question vector is used to query the vector database for the top-k most semantically similar chunks
3. Those chunks are assembled into a `context` string
4. A prompt is constructed: system instruction + context + question
5. The prompt is sent to the LLM, which generates an answer grounded in the retrieved context

The fundamental insight: the question vector and the vectors of relevant chunks point in similar directions, even when the question and the document use entirely different words. "Who invented

the telephone?" and "Alexander Graham Bell received a patent in 1876" are semantically related despite sharing no keywords. Semantic search finds that match; keyword search would miss it.

7.3 Grounding and Hallucination Reduction

Beyond overcoming context window limits, the most important benefit of RAG is **grounding** — constraining the model's answers to verified source material.

Without RAG, if an LLM doesn't know the answer to a question, it often generates a plausible-sounding but incorrect response — hallucination. With RAG and a strict system prompt ("Answer using only the context provided below. Say 'I don't know' if the information is not there"), the model is constrained to information you've verified exists in your documents.

If the relevant chunk isn't in the retrieved context, the model says it doesn't know — which is the correct, honest behaviour. This transforms RAG from a knowledge retrieval tool into a trust and reliability mechanism.

The system prompt instruction matters enormously. "Answer based only on the context" is not a suggestion — when the model is instructed clearly and the context is well-retrieved, compliance is very high. Weak or absent grounding instructions are one of the most common causes of hallucination in RAG systems.

7.4 Source Citations

Because RAG retrieves specific chunks with known source metadata (file name, page number, section heading), you can display exactly where each answer came from. This is one of the most valued features of production RAG systems — users can verify answers independently.

`RunnableParallel` makes citation retrieval clean: it runs the answer generation chain and the retriever simultaneously and returns both.

```
chain = RunnableParallel(
    answer=rag_answer_subchain,
    sources=retriever,
)
result = chain.invoke("What is the refund policy?")
# result["sources"] contains Document objects with metadata["source"] and
metadata["page"]
```

The metadata on each Document tells you which file and which page were used to generate the answer. These become the citation links displayed below the response.

7.5 Retrieval Strategies

Default similarity (top-k): Return the k most similar chunks. Works well for focused, specific questions. The risk: if you set k too high, later chunks may be weakly related and introduce noise into the context.

Score threshold retrieval: Add a minimum similarity cutoff. If no chunk scores above the

threshold, no context is returned and the model says it doesn't know — rather than being given weakly related text and confabulating an answer. This is the right strategy when "I don't know" is a better outcome than a low-confidence answer.

MMR (Maximal Marginal Relevance): Balances relevance and diversity. Without MMR, if your document contains five nearly identical paragraphs on the same topic, all five might appear in the top-k, wasting context window space with redundant content. MMR explicitly penalises chunks that are similar to already-selected chunks, ensuring the retrieved context covers the topic broadly rather than repeating the same points.

The right strategy depends on the application:

- Precise factual Q&A with clear failure mode → score threshold
- Broad topic questions needing comprehensive coverage → MMR
- General purpose → default similarity with well-tuned k

7.6 Conversational RAG

A plain RAG system is stateless — each question is retrieved and answered independently, with no memory of previous turns. This breaks for follow-up questions.

If a user asks "Tell me about the refund policy" and then asks "Does it apply to digital purchases?" — the second question, processed independently, retrieves chunks about product descriptions, not about refund policies, because "digital purchases" without refund context points in the wrong direction.

Conversational RAG adds a rephrasing step: before retrieval, use the LLM to rewrite the follow-up question as a self-contained question, using the conversation history to resolve pronouns and implicit references.

"Does it apply to digital purchases?" + history context → "Does the Nova Store refund policy apply to digital purchases?"

This rephrased question is then used for retrieval, and retrieval quality matches what you'd achieve if the user had asked the complete, explicit question from the start.

The architecture is two sequential LLM calls:

1. Rephrasing chain: (history, follow-up question) → standalone question
2. Standard RAG chain: standalone question → answer with citations

The extra LLM call costs a small amount of latency and tokens but dramatically improves retrieval quality for multi-turn conversations.

7.7 Limitations of RAG

Understanding what RAG cannot do is as important as knowing what it can.

Retrieval ceiling: If the relevant chunk isn't in the top-k retrieved results, the model cannot use it.

Retrieval errors don't produce "I don't know" — they produce confident answers based on whatever *was* retrieved. Improving retrieval quality — through better chunking, better embedding models, better query formulation — is often more impactful than improving the generative model.

Chunking sensitivity: The same document chunked differently produces different retrieval results. A sentence split across a chunk boundary may be incomplete in both chunks. Chunk size, overlap, and splitting strategy require empirical tuning for each type of document.

Cross-chunk synthesis: RAG retrieves isolated chunks. Questions requiring information synthesised across many separate sections — "List every product mentioned in this document" — require reading more broadly than top-k retrieval allows. This requires architectural additions like map-reduce patterns or iterative retrieval.

Embedding model consistency: The same embedding model must be used at both indexing and query time. Mixing models produces vectors in incompatible spaces where geometric proximity is meaningless.

---

8. Deployment

8.1 Local Development vs Live Deployment

Running `streamlit run app.py` on your laptop is invisible to anyone else. For others to use the app, they would need Python installed, all packages installed, your code, and your API key — an impractical requirement for end users.

Deployment means running the app on a permanently accessible server, reachable via a URL, from any device. When deployed, anyone with the link can use the app with no setup on their end.

The three challenges deployment introduces:

1. **Dependencies** — the server must know which packages to install
2. **Secrets** — API keys must reach the server without being exposed in code or git history
3. **Code delivery** — the server must receive your code and stay up to date automatically

Streamlit Community Cloud addresses all three in a way that requires minimal infrastructure knowledge.

8.2 requirements.txt

`requirements.txt` is a plain text file, one package per line, specifying every dependency your application needs. The deployment server reads this file and installs every listed package before starting your app. Without it, every import in your code will fail with `ModuleNotFoundError`.

A common mistake is forgetting packages that were installed so long ago locally that you've forgotten they're there. The reliable way to build `requirements.txt` is: activate a fresh virtual environment, install only what your app needs, then run `pip freeze > requirements.txt`.

Version pinning with >= (minimum version) rather than exact == versions is generally recommended for deployment: it allows the server to install the latest compatible version, picking up security patches and minor fixes without breaking your application.

8.3 Secrets Management

Your `.env` file is never deployed. It's in `.gitignore`, which means git ignores it, which means it never gets pushed to GitHub, which means Streamlit Cloud never sees it. This is correct — you never want API keys in your code repository.

Streamlit Community Cloud provides its own secrets management: you paste secrets into the app's settings panel in the Streamlit Cloud dashboard. They are stored encrypted and exposed to your running app through `st.secrets`.

The code pattern that works in both environments:

```
load_dotenv()  # reads .env locally; no-op on cloud
GEMINI_API_KEY = st.secrets.get("GEMINI_API_KEY", os.getenv("GEMINI_API_KEY",
""))
```

On the cloud, `st.secrets.get()` finds the key in Streamlit's encrypted store. Locally, it finds nothing there, falls back to `os.getenv()`, which reads from `.env` via `load_dotenv()`. One code path handles both environments cleanly.

The non-negotiable rule: A secret must never appear in any file committed to a git repository. Not in a comment, not as a "test value," not in a configuration file. GitHub's automated secret scanning will flag it and notify you, but by then it may already have been indexed and cached by third parties. Rotate any key that was ever committed, even for a moment.

8.4 .gitignore

`.gitignore` is a file that tells git which files and folders to exclude from version control. It is read before every staging operation (`git add`) and prevents matched files from being tracked.

The file uses glob patterns. A trailing `/` matches a directory and everything inside it. `*.pyc` matches all files with that extension.

Critical entries for an AI application:

```
.env                # API keys and secrets
.venv/              # virtual environment (hundreds of MB, fully reproducible)
__pycache__/        # Python bytecode cache
rag_db/             # ChromaDB on-disk data (large, not useful to others)
.DS_Store           # Mac OS folder metadata
Thumbs.db           # Windows thumbnail cache
```

Best practice: Create `.gitignore` before writing any code, before your first commit. Add `.env` to it before creating the `.env` file. A `.gitignore` that is added after a file is already tracked does not retroactively untrack that file — you must explicitly run `git rm --cached <file>` to stop tracking something already in git's history.

8.5 The GitHub-Streamlit Deployment Workflow

Streamlit Community Cloud connects directly to a GitHub repository. When you first deploy, you specify which repository, branch, and Python file to use. From that point forward:

- You edit code locally
- Test with `streamlit run app.py`
- Push to GitHub with `git push`
- Streamlit Cloud detects the push and automatically rebuilds and redeploys

There are no manual deployment steps after the initial setup. No FTP, no SSH, no server commands. The connection between your code and the live application is fully automated through git.

This also means deployment-related errors show up clearly: if a required package is missing from `requirements.txt`, the deployment fails at the install step with a specific error you can read in the build logs.

8.6 Common Deployment Failures

`ModuleNotFoundError: No module named 'pypdf'` A package used in the code is not listed in `requirements.txt`. Add it, push, and Streamlit Cloud redeploys automatically.

`GEMINI_API_KEY is not set` The secret was not added in Streamlit Cloud's secrets panel, or the key name has a typo. `GEMINI_API_KEY` and `Gemini_API_Key` are different keys. Check the spelling exactly.

`AttributeError` or unexpected import errors after a working deploy A dependency version conflict has arisen as packages update. Pin the problematic package to an exact version in `requirements.txt` and test locally.

App deploys but crashes when a PDF is uploaded Often a dependency issue with `pypdf` or `langchain_community`. Check the Streamlit Cloud logs for the exact traceback — the specific line and package are always shown.

8.7 Why Ollama Cannot Be Deployed on Streamlit Cloud

Ollama works by running a model server on your machine, listening on `http://localhost:11434`. When LangChain calls `ChatOllama(model="phi3:mini")`, it makes an HTTP request to that local server.

On Streamlit Community Cloud, there is no Ollama server. You cannot install it (no root access on shared cloud infrastructure), you cannot run it (cloud containers have no GPU and insufficient RAM for most models), and even if you could, the URL `localhost:11434` refers to the container's own loopback — not a running Ollama process.

For cloud deployment, model calls must go through an API: Gemini, OpenAI, Anthropic, or Hugging Face Inference API. LangChain's uniform interface makes swapping providers a one-line change — the same chain that uses `ChatOllama` locally uses `ChatGoogleGenerativeAI` in production, with everything else unchanged.

---

9. Dockerize the PDF Chat Application on Windows

- **9.1 Objective**

The objective is to containerize the existing pdf-chat-app Streamlit application using Docker on Windows. The application will run locally inside a Docker container without affecting the existing deployment on Streamlit Community Cloud.

Docker packages the application, its Python runtime, and its dependencies into an image. This image can be used to run the application consistently on different computers and, later, on cloud platforms.

The application repository is available at:

https://github.com/RakeshdotKR/pdf-chat-app

The following steps cover Docker installation, project setup, API key configuration, image building, container execution, testing, and troubleshooting.

- **9.2 Prerequisites**

The following software and resources are required:

A Windows 10 or Windows 11 computer that meets Docker Desktop's system requirements.

Administrator access to install software and enable required Windows features.

Git for downloading the project.

Docker Desktop for Windows.

A valid Gemini API key.

An internet connection for downloading dependencies and accessing the Gemini API.

Git can be downloaded from:

https://git-scm.com/download/win

Docker Desktop can be downloaded from:

https://www.docker.com/products/docker-desktop/

- **9.3 Install Docker Desktop on Windows**
- **9.3.1 Download Docker Desktop**

1. Open the official Docker Desktop download page:
2. https://www.docker.com/products/docker-desktop/
3. Download the Windows installer.
4. Run the downloaded installer.

5. Accept the installation prompts.

- **9.3.2 Configure the installation**

During installation, select the option to use the WSL 2 backend, if offered and supported by your system.

WSL 2 (Windows Subsystem for Linux version 2) provides the Linux environment commonly used to run Linux containers on Windows.

Complete the installation and restart Windows if requested.

- **9.3.3 Install or update WSL 2**

Open PowerShell as Administrator and execute:

wsl --install

If WSL is already installed, check its status:

wsl --status

wsl --version

If required, update WSL:

wsl --update

Restart the computer if Windows requests it.

If wsl --install reports that WSL is already installed, continue after confirming that the installation is working. If the computer does not support the WSL 2 backend, consult Docker's official Windows installation requirements.

- **9.3.4 Start Docker Desktop**

1. Open Docker Desktop from the Windows Start menu.
2. Accept the required terms and complete the initial setup.
3. Wait for Docker Desktop to finish starting.
4. Confirm that the Docker engine is running.

Docker Desktop provides the Docker engine and command-line tools needed to build and run containers.

- **9.3.5 Verify the installation**

Open a new PowerShell window and run:

docker --version

docker info

git --version

The first command displays the installed Docker version. The second displays information about the Docker engine. The third checks whether Git is installed.

Next, test Docker by running:

docker run hello-world

Docker downloads a small test image if necessary, runs it, and displays a confirmation message.

If this command succeeds, Docker is ready for the application setup.

- **9.4 Download the PDF Chat Application**
- **9.4.1 Open PowerShell**

Open PowerShell from the Windows Start menu.

Navigate to the home directory:

cd $HOME

- **9.4.2 Clone the GitHub repository**

Run:

git clone https://github.com/RakeshdotKR/pdf-chat-app.git

Navigate to the project directory:

cd pdf-chat-app

If the repository already exists on your computer, navigate to its directory instead of cloning it again.

- **9.4.3 Inspect the project files**

Execute:

Get-ChildItem

Display the dependency list:

Get-Content requirements.txt

The project should contain app.py and requirements.txt. The first file contains the Streamlit application, while the second lists the Python packages needed to run it.

The Docker configuration files will be added to this directory.

- **9.5 Create the Dockerfile**
- **9.5.1 Open the project in an editor**

If Visual Studio Code is installed, run:

code .

Otherwise, open the project directory in File Explorer and use a text editor.

- **9.5.2 Create the Dockerfile**

Create a file named exactly Dockerfile, without a .txt extension, in the project root.

Add the following content:

```
# Use a lightweight Python base image.
FROM python:3.12-slim

# Set the application directory inside the container.
WORKDIR /app
```

```
# Copy dependency information first.
COPY requirements.txt .

# Install the Python dependencies.
RUN python -m pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Copy the Streamlit application.
COPY app.py .

# Document the Streamlit port.
EXPOSE 8501

# Check whether the Streamlit server is responding.
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8501/_stcore/health',
timeout=3)" || exit 1
```

Start the Streamlit application.

CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]

- **9.5.3 Understand the Dockerfile**
  - FROM specifies the Python base image.
  - WORKDIR defines the working directory inside the container.
  - COPY transfers files from the project directory into the image.
  - RUN installs the dependencies listed in requirements.txt.
  - EXPOSE documents the port used by Streamlit.
  - HEALTHCHECK checks whether the application responds to health requests.
  - CMD starts the Streamlit server when the container runs.

The 0.0.0.0 address allows the application to be accessed through the port published by Docker.

Python 3.12 is the initial choice for this example. If a dependency in requirements.txt is incompatible with it, the Python version may need to be adjusted based on the actual build error.

- **9.6 Create the .dockerignore File**
- **9.6.1 Create the file**

In the project root, create a file named .dockerignore.

Add:

```
.git
.venv
venv
__pycache__
*.pyc
```

```
.env
.streamlit/secrets.toml
chroma_db
```

- **9.6.2 Understand its purpose**

The .dockerignore file prevents unnecessary local files from being sent to Docker during the image build.

It excludes Git history, local Python environments, temporary Python files, environment files, Streamlit secrets, and the specified local ChromaDB directory.

Excluding local data also avoids unintentionally including machine-specific vector-store data in the image.

- **9.7 Configure the Gemini API Key**
- **9.7.1 Obtain the API key**

If you do not already have a Gemini API key, visit:

https://aistudio.google.com/apikey

Follow the instructions to create or obtain a key for your Google account.

- **9.7.2 Understand API key handling**

The application requires a valid Gemini API key to generate embeddings and answer questions using the configured Google AI services.

The key must not be hardcoded in the Dockerfile, committed to GitHub, or included in the Docker image.

For the local Docker run, the key will be passed through an environment variable.

- **9.7.3 Set the API key in PowerShell**

Run:

```
$env:GEMINI_API_KEY = "YOUR_REAL_GEMINI_API_KEY"
```

Replace the placeholder with the actual key.

Verify that the variable exists without displaying its value:

```
if ($env:GEMINI_API_KEY) {
    Write-Output "Gemini API key is set"
} else {
    Write-Output "Gemini API key is missing"
}
```

The environment variable is available to commands run from that PowerShell session. If you open another PowerShell window, set it again.

- **9.7.4 Check the application's API key initialization**

The application should read the environment variable before attempting to access Streamlit secrets.

If necessary, replace the existing API key initialization in app.py with:

Read the key from the environment first.

```
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "")

# Fall back to Streamlit secrets if needed.
if not GEMINI_API_KEY:
    try:
        GEMINI_API_KEY = st.secrets.get("GEMINI_API_KEY", "")
    except Exception:
        GEMINI_API_KEY = ""

# Stop if no key is configured.
if not GEMINI_API_KEY:
    st.error("GEMINI_API_KEY is not configured.")
    st.stop()


# Configure the Google GenAI SDK.

os.environ["GOOGLE_API_KEY"] = GEMINI_API_KEY
```

Ensure that os and streamlit as st are imported in the file. Preserve the existing SDK configuration and other application logic.

This arrangement supports environment-variable configuration in Docker and a fallback to Streamlit secrets in environments where they are configured.

- **9.8 Build the Docker Image**
- **9.8.1 Verify the project structure**

Run:
Get-Location
Get-ChildItem
The directory should contain:

```
pdf-chat-app/
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

Additional project files may also be present.

- **9.8.2 Build the image (ensure Docker Desktop if running)**

Run the following command from the project root:

```
docker build -t pdf-chat-app:local .
```

The command instructs Docker to build an image using the Dockerfile in the current directory and assign it the name pdf-chat-app with the tag local.
During the build, Docker:

1. Downloads the Python base image if it is not already available.
2. Copies requirements.txt into the image.
3. Installs the Python dependencies.
4. Copies app.py into the image.
5. Creates the final image.

The first build may take several minutes because the application uses multiple Python libraries.

- **9.8.3 Verify the image**

After the build completes, run:

```
docker image ls pdf-chat-app
```

The output should show the pdf-chat-app image with the local tag.

If the build fails, retain the error output before making changes. The error will help identify whether the problem is related to dependency installation, Python compatibility, network access, or Docker configuration.

- **9.9 Run the Docker Container**
- **9.9.1 Start the container**

Make sure the Gemini API key is set in the current PowerShell session.

Run:

```
docker run --rm -p 8501:8501 -e GEMINI_API_KEY --name pdf-chat-app pdf-chat-app:local
```

The options have the following purposes:

- --rm removes the stopped container automatically.
- -p 8501:8501 maps port 8501 on Windows to port 8501 inside the container.
- -e GEMINI_API_KEY passes the existing environment variable into the container.
- --name pdf-chat-app assigns a recognizable name to the container.
- pdf-chat-app:local identifies the image to run.

The terminal displays Streamlit's startup messages. Keep the terminal open while testing the application.

- **9.9.2 Open the application**

Open a web browser and navigate to:

http://localhost:8501

The PDF Chat application should appear.

The container is running locally on your computer. This does not automatically make the application publicly available on the internet.

- **9.9.3 Stop the container**

When testing is complete, return to the PowerShell terminal running the container and press Ctrl+C.

Because the container was started with --rm, Docker removes the stopped container automatically.

The built image remains available unless it is explicitly removed.

- **9.10 Test the Application**
- **9.10.1 Test the user interface**

Confirm that the Streamlit page loads without errors.

- **9.10.2 Test PDF upload**

Upload a sample PDF and click the application's PDF processing button.

Confirm that the application processes the document successfully.

- **9.10.3 Test question answering**

Enter a question whose answer is present in the PDF.

Confirm that the application retrieves relevant content and returns a response through the configured Gemini model.

- **9.10.4 Test the health endpoint**

While the container is running, open:
http://localhost:8501/_stcore/health
A healthy Streamlit server should return a successful response.

- **9.10.5 Test completion checklist**
  - Docker image builds successfully.
  - Container starts without errors.
  - Streamlit interface opens at localhost:8501.
  - PDF upload and processing work.
  - Gemini generates a response.
  - The health endpoint responds successfully.
- **9.11 Troubleshooting Common Errors**
- **9.11.1 Docker engine unavailable**

Possible message: Cannot connect to the Docker daemon or Docker engine.
Solution:

1. Start Docker Desktop.
2. Wait until the engine is running.
3. Run docker info again.
4. If necessary, inspect the WSL status using wsl --status.

- **9.11.2 Dockerfile not found**

Possible message: Failed to read the Dockerfile.
Solution:
Run:
Get-Location
Get-ChildItem
Confirm that the current directory contains a file named Dockerfile. Make sure Windows has not saved it as Dockerfile.txt.
Run the build command from the project root.

- **9.11.3 Dependency installation failure**

Possible message: No matching distribution found or a package failed to build.
Solution:

1. Read the first meaningful error in the build output.
2. Check whether the package name and version in requirements.txt are valid.
3. Check whether the package supports Python 3.12.
4. Change the Python base image only if the error indicates a compatibility problem.
5. Rebuild the image after making a justified correction.

- **9.11.4 ModuleNotFoundError**

Possible message: A Python module cannot be found.
Solution:
Confirm that the missing package is listed in requirements.txt. Add it if necessary and rebuild:
docker build -t pdf-chat-app:local .

- **9.11.5 Gemini API key missing**

Possible message: GEMINI_API_KEY is not configured.
Solution:
Set the variable in the same PowerShell session:
$env:GEMINI_API_KEY = "YOUR_REAL_GEMINI_API_KEY"

Then start the container again.

- **9.11.6 Gemini authentication or quota error**

Possible message: HTTP 401, HTTP 403, or a quota-related error.
Solution:
Check that the API key is valid, the required API access is enabled, and the account's quota has not been exceeded.

- **9.11.7 Port 8501 already in use**

Possible message: Bind for 0.0.0.0:8501 failed because the port is already allocated.
Solution:
Use another host port:

```
docker run --rm -p 8502:8501 `
  -e GEMINI_API_KEY `
  --name pdf-chat-app `
  pdf-chat-app:local
```

Open:
http://localhost:8502
The application still listens on port 8501 inside the container. Only the host-side port has changed.

- **9.11.8 Container exits unexpectedly**

Inspect the containers:

```
docker ps -a
```

Inspect the logs:

```
docker logs pdf-chat-app
```

Use the logs to identify whether the failure occurred during application startup, dependency loading, configuration, or API communication.

- **9.12 Useful Docker Commands**

Command	Purpose
docker --version	Displays the Docker version.
docker info	Checks the Docker engine.
docker images	Lists local images.
docker ps	Lists running containers.
docker ps -a	Lists all containers, including stopped ones.
docker logs pdf-chat-app	Displays container logs.
docker stop pdf-chat-app	Stops a running container.
docker image rm pdf-chat-app:local	Removes the image when it is no longer needed.

- **9.13 Important Considerations**
- **9.13.1 Existing Streamlit deployment**

  The current Streamlit deployment can remain active while the Docker version is tested locally. Changes to the local Docker setup do not automatically affect the hosted application.

- **9.13.2 Persistent storage**

Files and data written only inside a container may not survive its removal. The current docker run command uses --rm, so stopped containers are removed.

For persistent ChromaDB storage, uploaded files, or other application data, configure an appropriate persistent volume or external storage service. The current Dockerfile does not provide persistent storage by itself.

- **9.13.3 API costs and security**

Docker does not eliminate Gemini API usage costs. Embedding generation and chat requests may incur charges according to the applicable Google API plan.

Keep API keys outside the image and source repository. For a public deployment, also consider authentication, upload-size limits, HTTPS, resource limits, and appropriate secrets management.

- **9.13.4 Future cloud deployment**

Once the application runs successfully in Docker, the image can serve as the basis for deployment to a Docker-compatible platform such as Amazon ECS with AWS Fargate, Amazon EC2, or another container hosting service.

A production deployment will require additional decisions about persistent storage, networking, secrets, monitoring, and scaling.

- **9.14 Conclusion**

The PDF Chat application can be containerized without redesigning its Streamlit interface. Docker Desktop provides the local container runtime, the Dockerfile defines the application environment, and the Docker image packages the application and its dependencies.

After successfully building and testing the image on Windows, the same containerization approach can be used as the foundation for a cloud deployment.

Quick Reference

The Conceptual Stack

```
Raw Text  →  Tokenise  →  Embed  →  Vector DB   (indexing)
Question  →  Embed     →  Retrieve  →  Context    (retrieval)
Context + Question  →  Prompt  →  LLM  →  Answer  (generation)
Answer + Sources  →  Streamlit UI  →  User         (presentation)
```

Key Concepts

Concept	One-line summary
Token	Atomic unit the LLM processes; ~4 English characters
Context window	Max tokens visible to the model in one call
Temperature	Sampling randomness; 0 = deterministic, 1+ = creative
Embedding	A vector encoding text meaning; similar text → similar vectors
Vector database	Stores embeddings; enables fast semantic search
Cosine similarity	Measures angle between vectors; 1.0 = identical direction
RAG	Retrieve relevant chunks, inject into prompt, LLM answers
Grounding	Constraining the model to provided context, reducing hallucination
LCEL	LangChain's `|` pipe syntax for composing chains
Session state	Streamlit's per-session persistence across re-runs
st.secrets	Streamlit Cloud's encrypted secrets management
Hallucination	Model confidently producing factually incorrect content

Model Selection

Use	Model
Text generation (cloud)	`gemini-3.8-flash` via `ChatGoogleGenerativeAI`
Text generation (local, fast)	`phi3:mini` via `ChatOllama`
Text generation (local, quality)	`gemma2:2b` via `ChatOllama`
Embeddings (LangChain)	`models/text-embedding-004` via `GoogleGenerativeAIEmbeddings`

Python 3.13+ | Gemini gemini-3.8-flash | Ollama phi3:mini / gemma2:2b | LangChain | ChromaDB | Streamlit